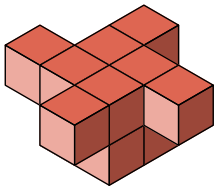
November 2, 2026 at 19:43

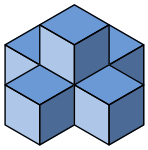
1. Introduction. Exercise 7.2.2.1–387 asks how many symmetry types a polycube can have, and for an example of each one "using the minimum number of cubies." Reflections are not allowed—a left twist is not a right twist—so the group in play is the 24 rotations of a cube rather than all 48 of its symmetries.

Answer 387 works it out: the 24 rotations are the signed permutations of $\{\pm1, \pm2, \pm3\}$ whose inversions and complementations together number an even amount; that group has 30 subgroups; the subgroups fall into 11 conjugacy classes; and those eleven classes are the symmetry types. It names them *full*, *even*, *8-fold*, *6-fold*, *90°*, *bidiagonal*, *tricentral*, *120°*, *diagonal*, *axial*, *none*, and draws a polycube for each.

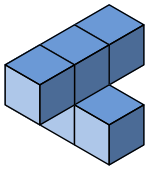
This program checks all of that, and then works out the minimum size of each type—which the answer states in words for only one of the eleven. Two of the eleven pictures turn out to be bigger than they need to be, and the one below is the clearer of the two.



answer 387's type (v): at least twelve



six of them will do



and six for type (vi)

```
package main
import (
    "flag"
    "fmt"
    "sort"
    "strings"
)
⟨ Types 4 ⟩
⟨ Functions 5 ⟩
func main() {
    ⟨ Read the command line 2 ⟩
    ⟨ Set up the two groups 7 ⟩
    ⟨ Do what the mode asks 3 ⟩
}
```

2. ⟨ Read the command line 2 ⟩ ≡

```
mode := flag.String("mode", "all", "group␣types␣grow␣min␣printed␣or␣all")
top := flag.Int("top", 8, "largest␣polycube␣to␣grow")
lim := flag.Int("lim", 24, "largest␣polycube␣the␣orbit␣search␣will␣look␣for")
flag.Parse()
```

This code is used in section 1.

3. ⟨ Do what the mode asks 3 ⟩ ≡

```
if *mode ≡ "group" ∨ *mode ≡ "all" {
    ⟨ Check the 24 rotations 8 ⟩
}
if *mode ≡ "types" ∨ *mode ≡ "all" {
    ⟨ Count the subgroups and the symmetry types 19 ⟩
    ⟨ Check what the answer says about particular types 21 ⟩
}
if *mode ≡ "grow" ∨ *mode ≡ "all" {
    ⟨ Grow every polycube and watch for new types 30 ⟩
}
if *mode ≡ "min" ∨ *mode ≡ "all" {
    ⟨ Find the smallest polycube of each type 36 ⟩
}
if *mode ≡ "printed" ∨ *mode ≡ "all" {
    ⟨ Weigh the pictures in answer 387 45 ⟩
}
```

This code is used in section 1.

4. The rotations of a cube. A signed permutation of the three coordinates is a matrix with one nonzero entry in each row and column, that entry being ±1; there are 48 of them, and the 24 with determinant +1 are the rotations. Answer 387 describes the same 24 differently—"the number of inversions of the permutation plus the number of complementations is even"—so the first thing to check is that the two descriptions pick out the same matrices.

⟨ Types 4 ⟩ ≡

```
type mat [3][3]int
type vec struct { x, y, z int }
```

This code is also defined in section 12.

This code is used in section 1.

5. ⟨ Functions 5 ⟩ ≡

```
func mul(a, b mat) mat {
    var c mat
    for i := range 3 {
        for j := range 3 {
            for k := range 3 {
                c[i][j] += a[i][k] * b[k][j]
            }
        }
    }
    return c
}
func inv(a mat) mat {   // these matrices are orthogonal, so this is the transpose
    var c mat
    for i := range 3 {
        for j := range 3 {
            c[i][j] = a[j][i]
        }
    }
    return c
}
func act(a mat, v vec) vec {
    c := [3]int{v.x, v.y, v.z}
    var r [3]int
    for i := range 3 {
        r[i] = a[i][0] * c[0] + a[i][1] * c[1] + a[i][2] * c[2]
    }
    return vec{r[0], r[1], r[2]}
}
func det(a mat) int {
    return a[0][0] * (a[1][1] * a[2][2] - a[1][2] * a[2][1]) -
        a[0][1] * (a[1][0] * a[2][2] - a[1][2] * a[2][0]) +
        a[0][2] * (a[1][0] * a[2][1] - a[1][1] * a[2][0])
}
```

This code is also defined in sections 6, 11, 13, 14, 15, 18, 20, 23, 27, 28, 29, 33, 35, 38, 40, 42, 44, 47, 52.

This code is used in section 1.

6. The 48 come out of the six permutations and the eight sign patterns. I keep the permutation and the signs alongside each matrix, because the answer's description is stated in terms of them.

⟨ Functions 5 ⟩ +≡

```
func signed( )(ms []mat, perms [][3]int, signs [][3]int) {
    for _, p := range [][3]int {{0, 1, 2}, {0, 2, 1}, {1, 0, 2},
        {1, 2, 0}, {2, 0, 1}, {2, 1, 0} }{
            for b := range 8 {
                var m mat
                var s [3]int
                for i := range 3 {
                    s[i] = 1 − 2 ∗ ((b ≫ i) & 1)
                    m[i][p[i]] = s[i]
                }
                ms = append(ms, m)
                perms = append(perms, p)
                signs = append(signs, s)
            }
        }
    return
}
```

7. ⟨ Set up the two groups 7 ⟩ ≡

```
ms, perms, signs := signed( )
var rot []mat
for _, m := range ms {
    if det(m) ≡ 1 {
        rot = append(rot, m)
    }
}
```

This code is also defined in section 16.

This code is used in section 1.

8. ⟨ Check the 24 rotations 8 ⟩ ≡

```
fmt.Println("the 24 rotations")
byDet := map[mat]bool{}
for _, m := range rot {
    byDet[m] = true
}
byRule := map[mat]bool{}
for i, m := range ms {
    ⟨ Count inversions and complementations of symmetry i 9 ⟩
    if (invs + comps) % 2 ≡ 0 {
        byRule[m] = true
    }
}
same := len(byDet) ≡ len(byRule)
for m := range byDet {
    if ¬byRule[m] {
        same = false
    }
}
fmt.Printf("  %d signed permutations, %d with determinant +1, %d with" +
    " inversions+complementations even; the two agree: %v\n",
    len(ms), len(byDet), len(byRule), same)
⟨ Check the rotation the answer works out 10 ⟩
```

This code is used in section 3.

9. ⟨ Count inversions and complementations of symmetry i 9 ⟩ ≡

```
p, s := perms[i], signs[i]
invs, comps := 0, 0
for a := range 3 {
    if s[a] < 0 {
        comps++
    }
    for b := a + 1; b < 3; b++ {
        if p[a] > p[b] {
            invs++
        }
    }
}
```

This code is used in section 8.

10. The answer picks one symmetry out for an example: $\bar{1}32$ takes $(x, y, z) \mapsto (c - x, z, y)$, a rotation of 180° about the line $x = c/2$, $y = z$; it is a symmetry of the bent tricube $\{000, 001, 010\}$ when $c = 0$, and of the L-twist $\{000, 001, 100, 110\}$ when $c = 1$. Four claims, all easy to try.

⟨ Check the rotation the answer works out 10 ⟩ ≡

```
bar132 := mat{{-1, 0, 0}, {0, 0, 1}, {0, 1, 0}}
tr := 0
for i := range 3 {
    tr += bar132[i][i]
}
var ax vec
for _, v := range []vec {{0, 1, 1}, {1, 0, 0}, {0, 1, -1} }{
    if act(bar132, v) ≡ v {
        ax = v
    }
}
fmt.Printf("␣␣the␣answer's␣example:␣determinant␣%d,␣trace␣%d␣(so␣a␣half␣turn)," +
    "␣axis␣(%d,%d,%d)\n", det(bar132), tr, ax.x, ax.y, ax.z)
bent := []vec{{0, 0, 0}, {0, 0, 1}, {0, 1, 0}}
twist := []vec{{0, 0, 0}, {0, 0, 1}, {1, 0, 0}, {1, 1, 0}}
fmt.Printf("␣␣it␣fixes␣the␣bent␣tricube␣with␣c=0:␣%v;" +
    "␣the␣L-twist␣with␣c=1:␣%v\n",
    fixes(bar132, vec{0, 0, 0}, bent), fixes(bar132, vec{1, 0, 0}, twist))
```

This code is used in section 8.

11. A map $x \mapsto Mx + t$ carries a set of cubies onto itself, or it does not.

⟨ Functions 5 ⟩ +≡

```
func fixes(m mat, t vec, p []vec) bool {
    in := map[vec]bool{ }
    for _, c := range p {
        in[c] = true
    }
    for _, c := range p {
        q := act(m, c)
        if ¬in[(vec{q.x + t.x, q.y + t.y, q.z + t.z})] {
            return false
        }
    }
    return true
}
```

12. Subgroups, and symmetry types. A subgroup of a group of at most 48 elements is a set of elements closed under the operation, so a bit vector is all the room one needs. Closing a set is a matter of multiplying everything by everything until nothing new appears, and the subgroups are what you reach by starting at the identity and throwing in one element at a time until the collection stops growing.

⟨ Types 4 ⟩ +≡

```
type grp uint64
```

13. ⟨ Functions 5 ⟩ +≡

```
func closure(el []mat, idx map[mat]int, g grp) grp {
    g |= 1 ≪ idx[mat{{1,0,0},{0,1,0},{0,0,1}}]
    for again := true; again; {
        again = false
        for i := range el {
            if g & (1 ≪ i) ≡ 0 {
                continue
            }
            for j := range el {
                if g & (1 ≪ j) ≡ 0 {
                    continue
                }
                k := grp(1) ≪ idx[mul(el[i], el[j])]
                if g & k ≡ 0 {
                    g |= k
                    again = true
                }
            }
        }
    }
    return g
}
func subgroups(el []mat, idx map[mat]int) []grp {
    have := map[grp]bool{closure(el, idx, 0): true}
    for again := true; again; {
        again = false
        for h := range have {
            for i := range el {
                k := closure(el, idx, h | (1 ≪ i))
                if ¬have[k] {
                    have[k] = true
                    again = true
                }
            }
        }
    }
    out := make([]grp, 0, len(have))
    for h := range have {
        out = append(out, h)
    }
    sort.Slice(out, func(a, b int) bool { return out[a] < out[b] })
    return out
}
```

14. Two subgroups are conjugate when one is $t^{-}Tt$ for some t in the whole group; the answer explains that conjugate subgroups amount to looking at the object from a different direction, so the symmetry types are the conjugacy classes.

⟨ Functions 5 ⟩ +≡

```
func conjugate(el []mat, idx map[mat]int, h grp, t mat) grp {
    var k grp
    ti := inv(t)
    for i := range el {
        if h & (1 << i) != 0 {
            k |= 1 << idx[mul(mul(ti, el[i]), t)]
        }
    }
    return k
}
func classesOf(el []mat, idx map[mat]int)(subs []grp, cls [][]grp) {
    subs = subgroups(el, idx)
    seen := map[grp]bool{}
    for _, h := range subs {
        if seen[h] {
            continue
        }
        var c []grp
        in := map[grp]bool{}
        for _, t := range el {
            k := conjugate(el, idx, h, t)
            if !in[k] {
                in[k] = true
                c = append(c, k)
            }
            seen[k] = true
        }
        cls = append(cls, c)
    }
    sort.Slice(cls, func(a, b int) bool {
        if x, y := bits(cls[a][0]), bits(cls[b][0]); x != y {
            return x > y
        }
        return len(cls[a]) < len(cls[b])
    })
    return
}
func bits(g grp) int {
    n := 0
    for; g != 0; g &= g - 1 {
        n++
    }
    return n
}
```

15. Each class gets the name answer 387 gives it. Order and number of conjugates tell the eleven apart except for the two classes of order 4 with three conjugates each, and those two differ in whether they contain a quarter turn.

⟨ Functions 5 ⟩ +≡

```
func nameOf(el []mat, h grp, conj int) string {
    quarter := false
    for i := range el {
        if h & (1 << i) ≠ 0 ∧ order(el, el[i]) ≡ 4 {
            quarter = true
        }
    }
    switch {
    case bits(h) ≡ 24:
        return "full"
    case bits(h) ≡ 12:
        return "even"
    case bits(h) ≡ 8:
        return "8-fold"
    case bits(h) ≡ 6:
        return "6-fold"
    case bits(h) ≡ 4 ∧ conj ≡ 1:
        return "tricentral"
    case bits(h) ≡ 4 ∧ quarter:
        return "90␣deg"
    case bits(h) ≡ 4:
        return "bidiagonal"
    case bits(h) ≡ 3:
        return "120␣deg"
    case bits(h) ≡ 2 ∧ conj ≡ 6:
        return "diagonal"
    case bits(h) ≡ 2:
        return "axial"
    }
    return "none"
}
func order(el []mat, m mat) int {
    id := mat{{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}
    k, a := 1, m
    for a ≠ id {
        a = mul(a, m)
        k++
    }
    return k
}
```

16. ⟨ Set up the two groups 7 ⟩ +≡

```
rotIdx := map[mat]int{}
for i, m := range rot {
    rotIdx[m] = i
}
fullIdx := map[mat]int{}
for i, m := range ms {
    fullIdx[m] = i
}
rotSubs, rotCls := classesOf(rot, rotIdx)
⟨ Number the classes the way answer 387 does 17 ⟩
```

17. Answer 387 numbers the eleven classes (i) to (xi) in an order of its own, so I put them in that order too and use its numbers throughout.

⟨ Number the classes the way answer 387 does 17 ⟩ ≡

```
sort.Slice(rotCls, func(a, b int) bool {
    return answerOrder(nameOf(rot, rotCls[a][0], len(rotCls[a]))) <
        answerOrder(nameOf(rot, rotCls[b][0], len(rotCls[b])))
})
```

This code is used in section 16.

18. ⟨ Functions 5 ⟩ +≡

```
func answerOrder(n string) int {
    for i, s := range []string { "full", "even", "8-fold", "6-fold", "90␣deg",
        "bidiagonal", "tricentral", "120␣deg", "diagonal", "axial", "none" }{
            if s ≡ n {
                return i
            }
        }
    return 99
}
```

19. ⟨ Count the subgroups and the symmetry types 19 ⟩ ≡

```
fmt.Println("subgroups␣and␣symmetry␣types")
fmt.Printf("␣␣the␣24␣rotations␣have␣%d␣subgroups␣in␣%d␣conjugacy␣classes\n",
    len(rotSubs), len(rotCls))
fullSubs, fullCls := classesOf(ms, fullIdx)
fmt.Printf("␣␣all␣48␣signed␣permutations␣have␣%d␣subgroups␣in␣%d␣classes\n",
    len(fullSubs), len(fullCls))
for i, c := range rotCls {
    fmt.Printf("␣␣(%s)␣%-11s␣order␣%2d,␣%d␣conjugate%s\n",
        roman(i + 1), nameOf(rot, c[0], len(c)), bits(c[0]), len(c),
        map[bool]string{true: "", false: "s"}[len(c) ≡ 1])
}
```

This code is used in section 3.

20. ⟨ Functions 5 ⟩ +≡

```
func roman(n int) string {
    return []string{"", "i", "ii", "iii", "iv", "v", "vi", "vii", "viii",
        "ix", "x", "xi"}[n]
}
```

21. What the answer says about three of the types. Three sentences in answer 387 say something specific enough to test. Class (ii) "consists of the 12 symmetries whose permutations are even"; class (iv) "has one symmetry for each permutation of the three coordinates"; and seven of the classes "correspond to the eight symmetry types of a square, with reflections implemented by turning the square over," the square's 180° having become the same thing as *axial*.

```
⟨ Check what the answer says about particular types 21 ⟩ ≡
    fmt.Println("what␣the␣answer␣says␣about␣particular␣types")
    ⟨ Look at the class of order 12 22 ⟩
    ⟨ Look at the class of order 6 24 ⟩
    ⟨ Turn a square over 25 ⟩
```

This code is used in section 3.

22.

```
⟨ Look at the class of order 12 22 ⟩ ≡
    for _, c := range rotCls {
        if bits(c[0]) ≠ 12 {
            continue
        }
        evens, all := 0, true
        for i, m := range rot {
            e := parityOf(m) ≡ 0
            if c[0] & (1 ≪ i) ≠ 0 {
                if e {
                    evens++
                } else {
                    all = false
                }
            } else if e {
                all = false
            }
        }
        fmt.Printf("␣␣class␣(ii):␣%d␣symmetries,␣all␣with␣even␣permutations," +
            "␣and␣no␣others:␣%v\n", evens, all)
    }
```

This code is used in section 21.

23. The permutation part of a signed permutation is even or odd on its own, whatever the signs do.

⟨ Functions 5 ⟩ +≡

```
func parityOf(m mat) int {
    var p [3]int
    for i := range 3 {
        for j := range 3 {
            if m[i][j] ≠ 0 {
                p[i] = j
            }
        }
    }
    n := 0
    for a := range 3 {
        for b := a + 1; b < 3; b++ {
            if p[a] > p[b] {
                n++
            }
        }
    }
    return n % 2
}
```

24. ⟨ Look at the class of order 6 24 ⟩ ≡

```
for _, c := range rotCls {
    if bits(c[0]) ≠ 6 {
        continue
    }
    seen := map[[3]int]bool{}
    for i, m := range rot {
        if c[0] & (1 ≪ i) ≠ 0 {
            var p [3]int
            for a := range 3 {
                for b := range 3 {
                    if m[a][b] ≠ 0 {
                        p[a] = b
                    }
                }
            }
            seen[p] = true
        }
    }
    fmt.Printf("␣␣class␣(iv):␣%d␣symmetries␣covering␣%d␣of␣the␣6␣permutations\n",
        bits(c[0]), len(seen))
}
```

This code is used in section 21.

25. A square that may be turned over has the eight symmetries of a square prism: four rotations about the prism's axis, and four half turns about axes across it. Its subgroups fall into eight conjugacy classes, which are the eight symmetry types of a polyomino; each of them sits inside one of the eleven classes of the whole rotation group, and the question is how many different ones they land in.

```
⟨ Turn a square over 25 ⟩ ≡
    var d4 []mat
    for _, m := range rot {
        if act(m, vec{0, 0, 1}) ≡ (vec{0, 0, 1}) ∨ act(m, vec{0, 0, 1}) ≡ (vec{0, 0, −1}) {
            d4 = append(d4, m)
        }
    }
    d4Idx := map[mat]int{}
    for i, m := range d4 {
        d4Idx[m] = i
    }
    _, d4Cls := classesOf(d4, d4Idx)
    ⟨ Say where each type of square lands 26 ⟩
```

This code is used in section 21.

26.

```
⟨ Say where each type of square lands 26 ⟩ ≡
    where := map[string][]int{}
    for i, c := range d4Cls {
        var g grp
        for j := range d4 {
            if c[0] & (1 ≪ j) ≠ 0 {
                g |= 1 ≪ rotIdx[d4[j]]
            }
        }
        for k, rc := range rotCls {
            for _, h := range rc {
                if h ≡ g {
                    n := nameOf(rot, h, len(rc))
                    where[n] = append(where[n], i)
                    _ = k
                }
            }
        }
    }
    var names []string
    for n := range where {
        names = append(names, n)
    }
    sort.Strings(names)
    fmt.Printf("␣␣the␣square's␣%d␣types␣land␣in␣%d␣of␣the␣eleven␣classes:␣%s\n",
        len(d4Cls), len(where), strings.Join(names, ",␣"))
    for _, n := range names {
        if len(where[n]) > 1 {
            fmt.Printf("␣␣two␣of␣the␣square's␣types␣become␣the␣same␣class␣here:␣%s\n", n)
        }
    }
```

This code is used in section 25.

27. The symmetry type of a polycube. A rotation is a symmetry of a polycube when the rotated cubies, slid back into place, are the cubies one started with. Those rotations form a subgroup, and the class of that subgroup is the polycube's symmetry type.

⟨ Functions 5 ⟩ +≡

```
func shift(p []vec) []vec {
    lo := p[0]
    for _, c := range p {
        lo = vec{min(lo.x, c.x), min(lo.y, c.y), min(lo.z, c.z)}
    }
    out := make([]vec, len(p))
    for i, c := range p {
        out[i] = vec{c.x − lo.x, c.y − lo.y, c.z − lo.z}
    }
    sort.Slice(out, func(a, b int) bool {
        if out[a].x ≠ out[b].x {
            return out[a].x < out[b].x
        }
        if out[a].y ≠ out[b].y {
            return out[a].y < out[b].y
        }
        return out[a].z < out[b].z
    })
    return out
}
func key(p []vec) string {
    var b strings.Builder
    for _, c := range p {
        fmt.Fprintf(&b, "%d.%d.%d;", c.x, c.y, c.z)
    }
    return b.String()
}
```

28. ⟨ Functions 5 ⟩ +≡

```
func symgroup(rot []mat, rotIdx map[mat]int, p []vec) grp {
    want := key(shift(p))
    var g grp
    for i, m := range rot {
        q := make([]vec, len(p))
        for j, c := range p {
            q[j] = act(m, c)
        }
        if key(shift(q)) ≡ want {
            g |= 1 ≪ i
        }
    }
    return g
}
```

29. Two polycubes are the same when a rotation carries one to the other, so the smallest of the 24 turned-and-slid forms is a name they share.

⟨ Functions 5 ⟩ +≡

```
func canon(rot []mat, p []vec) string {
    best := ""
    for _, m := range rot {
        q := make([]vec, len(p))
        for j, c := range p {
            q[j] = act(m, c)
        }
        if k := key(shift(q)); best == "" ∨ k < best {
            best = k
        }
    }
    return best
}
func unkey(s string) []vec {
    var p []vec
    for _, t := range strings.Split(strings.TrimSuffix(s, ";"), ";") {
        var c vec
        fmt.Sscanf(t, "%d.%d.%d", &c.x, &c.y, &c.z)
        p = append(p, c)
    }
    return p
}
```

30. Growing all the polycubes. The plainest way to find the smallest polycube of a given type is to look at all the small ones. Grow them a cubie at a time, keep one representative of each rotation class, and note when a type first shows up.

```
⟨ Grow every polycube and watch for new types 30 ⟩ ≡
    fmt.Printf("growing every polycube up to %d cubies\n", *top)
    cur := map[string]bool{canon(rot, []vec{{0,0,0}}): true}
    firstAt := map[string]int{}
    firstIs := map[string]string{}
    for n := 1; n ≤ *top; n++ {
        if n > 1 {
            ⟨ Add one cubie in every way 31 ⟩
        }
        ⟨ Record any type seen for the first time 32 ⟩
        fmt.Printf("  %2d cubies: %6d polycubes\n", n, len(cur))
    }
    ⟨ Report the smallest polycube of each type 34 ⟩
```

This code is used in section 3.

```
31. ⟨ Add one cubie in every way 31 ⟩ ≡
    next := map[string]bool{}
    for s := range cur {
        p := unkey(s)
        in := map[vec]bool{}
        for _, c := range p {
            in[c] = true
        }
        for _, c := range p {
            for _, d := range []vec {{1,0,0},{−1,0,0},{0,1,0},
                {0,−1,0},{0,0,1},{0,0,−1} }{
                q := vec{c.x + d.x, c.y + d.y, c.z + d.z}
                if ¬in[q] {
                    next[canon(rot, append(append([]vec{}, p...), q))] = true
                }
            }
        }
    }
    cur = next
```

This code is used in sections 30, 46, 50.

```
32. ⟨ Record any type seen for the first time 32 ⟩ ≡
    for s := range cur {
        p := unkey(s)
        nm := typeName(rot, rotIdx, rotCls, p)
        if _, ok := firstAt[nm]; ¬ok {
            firstAt[nm], firstIs[nm] = n, s
        }
    }
```

This code is used in section 30.

33. ⟨ Functions 5 ⟩ +≡

```
func typeName(rot []mat, rotIdx map[mat]int, cls [][]grp, p []vec) string {
    g := symgroup(rot, rotIdx, p)
    for _, c := range cls {
        for _, h := range c {
            if h ≡ g {
                return nameOf(rot, h, len(c))
            }
        }
    }
    return "?"
}
```

34. ⟨ Report the smallest polycube of each type 34 ⟩ ≡

```
for i, c := range rotCls {
    nm := nameOf(rot, c[0], len(c))
    if n, ok := firstAt[nm]; ok {
        fmt.Printf("␣␣(%-4s␣%-11s␣%2d␣cubies:␣␣%s\n",
            roman(i + 1) + ")", nm, n, show(unkey(firstIs[nm])))
    } else {
        fmt.Printf("␣␣(%-4s␣%-11s␣none␣up␣to␣%d␣cubies\n",
            roman(i + 1) + ")", nm, ∗top)
    }
}
```

This code is used in section 30.

35. ⟨ Functions 5 ⟩ +≡

```
func show(p []vec) string {
    var b []string
    for _, c := range p {
        b = append(b, fmt.Sprintf("%d%d%d", c.x, c.y, c.z))
    }
    return strings.Join(b, "␣")
}
```

36. The minimum for every type. Growing polycubes runs out of room long before it reaches twenty cubies, so the type of order 12 needs a different handle. Here is one. Every symmetry of a polycube carries its bounding box onto the bounding box, and so fixes the centre of that box; the centre has half-integer coordinates, which become integers if every cubie coordinate is doubled. A polycube whose symmetries are G is therefore a union of orbits of G acting about that centre—and if it has n cubies, none of them is further than $n-1$ from the centre in doubled coordinates, because a connected polycube of n cubies spans at most n in any direction.

So: try each of the eight centres modulo 2, take every orbit that fits inside the box, and look for a set of orbits with n cubies between them that is connected and has exactly the symmetries asked for. Nothing is missed.

⟨ Find the smallest polycube of each type 36 ⟩ ≡

```
fmt.Println("the␣smallest␣polycube␣of␣each␣type")
for i, c := range rotCls {
    nm := nameOf(rot, c[0], len(c))
    var found []vec
    for n := 1; n ≤ *lim ∧ found ≡ Λ; n++ {
        found = smallest(rot, rotIdx, rotCls, c[0], n)
        if found ≠ Λ {
            fmt.Printf("␣␣(%-4s␣%-11s␣%2d␣cubies:␣␣%s\n",
                roman(i+1)+")", nm, n, show(found))
            if nm ≡ "even" {
                ⟨ Look for the core of eight answer 387 describes 37 ⟩
            }
        }
    }
    if found ≡ Λ {
        fmt.Printf("␣␣(%-4s␣%-11s␣nothing␣up␣to␣%d␣cubies\n",
            roman(i+1)+")", nm, *lim)
    }
}
```

This code is used in section 3.

37. Of the class of order 12 the answer says more: "the smallest polycube which admits these symmetries and no more ... contains 20 cubies, with 12 surrounding a central core of 8." A core of 8 can only be a $2\times2\times2$ block, so I look for one and count what is left over.

⟨ Look for the core of eight answer 387 describes 37 ⟩ ≡

```
core := 0
for _, c := range found {
    if c.x ≥ 1 ∧ c.x ≤ 2 ∧ c.y ≥ 1 ∧ c.y ≤ 2 ∧ c.z ≥ 1 ∧ c.z ≤ 2 {
        core++
    }
}
fmt.Printf("␣␣␣␣␣␣␣␣of␣those,␣%d␣form␣a␣2x2x2␣block␣and␣%d␣surround␣it\n",
    core, len(found) − core)
```

This code is used in section 36.

38. ⟨ Functions 5 ⟩ +≡

```
func smallest(rot []mat, rotIdx map[mat]int, cls [][]grp, g grp, n int) []vec {
    for c := range 8 {
        q := vec{c & 1, (c ≫ 1) & 1, (c ≫ 2) & 1}
        ⟨ Give up on this centre unless the group acts about it 39 ⟩
        orbs := orbitsAbout(rot, g, q, n)
        if p := pickOrbits(rot, rotIdx, cls, g, orbs, 0, n, Λ); p ≠ Λ {
            return p
        }
    }
    return Λ
}
```

39. The rotation m about q sends a cubie at x to $m(x-q)+q$, which lands on the cubie lattice for every x only when $(I-m)q$ is even in each coordinate.

⟨ Give up on this centre unless the group acts about it 39 ⟩ ≡

```
ok := true
for i, m := range rot {
    if g & (1 ≪ i) ≡ 0 {
        continue
    }
    d := act(m, q)
    if (q.x − d.x) % 2 ≠ 0 ∨ (q.y − d.y) % 2 ≠ 0 ∨ (q.z − d.z) % 2 ≠ 0 {
        ok = false
    }
}
if ¬ok {
    continue
}
```

This code is used in section 38.

40. ⟨ Functions 5 ⟩ +≡

```
func orbitsAbout(rot []mat, g grp, q vec, n int) [][]vec {
    r := n − 1
    seen := map[vec]bool{}
    var out [][]vec
    for x := q.x − r; x ≤ q.x + r; x++ {
        for y := q.y − r; y ≤ q.y + r; y++ {
            for z := q.z − r; z ≤ q.z + r; z++ {
                p := vec{x, y, z}
                ⟨ Collect the orbit of p, if it is new and it fits 41 ⟩
            }
        }
    }
    sort.Slice(out, func(a, b int) bool { return len(out[a]) < len(out[b]) })
    return out
}
```

41. Only points of the doubled lattice are cubies, and only an orbit that lies wholly inside the box can belong to a polycube of this size.

```
⟨ Collect the orbit of p, if it is new and it fits 41 ⟩ ≡
    if x % 2 ≠ 0 ∨ y % 2 ≠ 0 ∨ z % 2 ≠ 0 ∨ seen[p] {
        continue
    }
    var orb []vec
    in := map[vec]bool{}
    for i, m := range rot {
        if g & (1 ≪ i) ≡ 0 {
            continue
        }
        d := act(m, vec{p.x − q.x, p.y − q.y, p.z − q.z})
        w := vec{d.x + q.x, d.y + q.y, d.z + q.z}
        if ¬in[w] {
            in[w] = true
            orb = append(orb, w)
        }
    }
    fits := len(orb) ≤ n
    for _, w := range orb {
        seen[w] = true
        if w.x < q.x − r ∨ w.x > q.x + r ∨ w.y < q.y − r ∨ w.y > q.y + r ∨
            w.z < q.z − r ∨ w.z > q.z + r {
            fits = false
        }
    }
    if fits {
        out = append(out, orb)
    }
```

This code is used in section 40.

42. The orbits are sorted by size, so the walk can stop as soon as the next one is too big for what is left.

```
⟨ Functions 5 ⟩ +≡
    func pickOrbits(rot []mat, rotIdx map[mat]int, cls [][]grp, g grp,
        orbs [][]vec, from, left int, chosen []vec) []vec {
        if left ≡ 0 {
            ⟨ Report these orbits if they make the polycube we want 43 ⟩
        }
        for j := from; j < len(orbs); j++ {
            if len(orbs[j]) > left {
                break
            }
            if p := pickOrbits(rot, rotIdx, cls, g, orbs, j + 1,
                left − len(orbs[j]), append(chosen, orbs[j]...)); p ≠ Λ {
                return p
            }
        }
        return Λ
    }
```

43. ⟨ Report these orbits if they make the polycube we want 43 ⟩ ≡

```
p := make([]vec, len(chosen))
for i, c := range chosen {
    p[i] = vec{c.x/2, c.y/2, c.z/2}
}
if ¬connected(p) {
    return Λ
}
h := symgroup(rot, rotIdx, p)
for _, c := range cls {
    if sameClass(c, g) ∧ sameClass(c, h) {
        return shift(p)
    }
}
return Λ
```

This code is used in section 42.

44. ⟨ Functions 5 ⟩ +≡

```
func sameClass(c []grp, g grp) bool {
    for _, k := range c {
        if k ≡ g {
            return true
        }
    }
    return false
}
func connected(p []vec) bool {
    in := map[vec]bool{}
    for _, c := range p {
        in[c] = true
    }
    seen := map[vec]bool{p[0]: true}
    stack := []vec{p[0]}
    for len(stack) > 0 {
        c := stack[len(stack) − 1]
        stack = stack[:len(stack) − 1]
        for _, d := range []vec {{1, 0, 0}, {−1, 0, 0}, {0, 1, 0},
            {0, −1, 0}, {0, 0, 1}, {0, 0, −1} }{
            q := vec{c.x + d.x, c.y + d.y, c.z + d.z}
            if in[q] ∧ ¬seen[q] {
                seen[q] = true
                stack = append(stack, q)
            }
        }
    }
    return len(seen) ≡ len(p)
}
```

45. Weighing the pictures in answer 387. The exercise asks for an example of each type "using the minimum number of cubies," so the eleven drawings ought to be as small as the numbers above. Counting cubies in a small isometric drawing is not something to do by eye, but one feature of such a drawing can be counted without any risk: the shaded top faces, one for each cubie that has nothing directly above it. I read those off the printed figure by looking for connected patches of the colour reserved for top faces, and got

1	full
9	even
2	8-fold
3	6-fold
8	90°
5	bidiagonal
6	tricentral
3	120°
3	diagonal
4	axial
4	none.

So the most top faces an n-cubie polycube can ever show is the largest number of its cubies that have nothing directly above them, over the six directions it might be drawn with upwards; if a drawing shows more than that, the thing drawn is not of the minimum size.

My first version said instead that a drawing shows one top face for each column, and bounded the count by the largest of the three projections. That is not right in general: a column with a gap in it has two exposed tops, and an oblique drawing can show part of the lower one. I came back to this in November 2026, after a bug in another of these readings (exercise 334) had hidden behind an assumption that all its checks shared. For every type at its minimum size the two bounds turn out to be equal.

That recheck also sent me back to the figure itself, and two of the counts I had read off it in September were wrong: the even type shows 9 top faces, not 10, and the bidiagonal type 5, not 6. The 9 changes nothing. The 5 changes the argument for type (vi), because six cubies can show five top faces; but the picture still cannot be of six cubies, for another reason, given below.

⟨ Weigh the pictures in answer 387 45 ⟩ ≡

```
fmt.Println("weighing␣the␣pictures␣against␣the␣minimum")
printed := map[string]int{"full": 1, "even": 9, "8-fold": 2, "6-fold": 3,
    "90␣deg": 8, "bidiagonal": 5, "tricentral": 6, "120␣deg": 3,
    "diagonal": 3, "axial": 4, "none": 4}
⟨ Find, for each size, the most top faces a polycube of each type can show 46 ⟩
⟨ Say which pictures are too big 48 ⟩
⟨ Look at the type (v) picture itself 49 ⟩
⟨ Look at the type (vi) picture itself 50 ⟩
```

This code is used in section 3.

46. ⟨ Find, for each size, the most top faces a polycube of each type can show 46 ⟩ ≡

```
mostTops := map[string]map[int]int{}
cur := map[string]bool{canon(rot, []vec{{0,0,0}}): true}
for n := 1; n ≤ 6; n++ {
    if n > 1 {
        ⟨ Add one cubie in every way 31 ⟩
    }
    for s := range cur {
        p := unkey(s)
        nm := typeName(rot, rotIdx, rotCls, p)
        t := exposedTops(p)
        if mostTops[nm] ≡ Λ {
            mostTops[nm] = map[int]int{}
        }
        if t > mostTops[nm][n] {
            mostTops[nm][n] = t
        }
    }
}
```

This code is used in section 45.

47. ⟨ Functions 5 ⟩ +≡

```
func exposedTops(p []vec) int {
    in := map[vec]bool{}
    for _, c := range p {
        in[c] = true
    }
    best := 0
    for _, u := range []vec {{1,0,0},{−1,0,0},{0,1,0},{0,−1,0},
        {0,0,1},{0,0,−1} }{
            k := 0
            for _, c := range p {
                if ¬in[vec{c.x + u.x, c.y + u.y, c.z + u.z}] {
                    k++
                }
            }
            if k > best {
                best = k
            }
        }
    return best
}
```

48. ⟨ Say which pictures are too big 48 ⟩ ≡

```
for i, c := range rotCls {
    nm := nameOf(rot, c[0], len(c))
    least := 0
    for n := 1; n ≤ 6; n++ {
        if mostTops[nm] ≠ Λ ∧ mostTops[nm][n] > 0 ∧ least ≡ 0 {
            least = n
        }
    }
    if least ≡ 0 {
        fmt.Printf("␣␣(%-4s␣%-11s␣printed␣drawing␣shows␣%2d␣top␣faces;" +
            "␣minimum␣size␣is␣past␣the␣reach␣of␣this␣check\n",
            roman(i + 1) + ")", nm, printed[nm])
        continue
    }
    verdict := "consistent␣with␣the␣minimum"
    if printed[nm] > mostTops[nm][least] {
        verdict = fmt.Sprintf("TOO␣BIG:␣%d␣cubies␣can␣show␣at␣most␣%d",
            least, mostTops[nm][least])
    }
    fmt.Printf("␣␣(%-4s␣%-11s␣minimum␣%d,␣which␣can␣show␣at␣most␣%d␣top" +
        "␣faces;␣the␣drawing␣shows␣%2d␣--␣%s\n",
        roman(i + 1) + ")", nm, least, mostTops[nm][least], printed[nm], verdict)
}
```

This code is used in section 45.

49. The type (v) drawing's eight top faces all sit at one height in the drawing's projection, and they make a pinwheel. In September I took that to mean a flat pinwheel of eight cubies. It does not: looked at again, the drawing also shows front faces below that level, so there is more underneath. Every cubie of the top level shows its top, so that level is the pinwheel exactly; and the quarter turn of a pinwheel is about an axis through the corner shared by its middle four cells, which passes through no cubie, so whatever lies below comes in fours. The drawing therefore has at least twelve cubies. The likeliest reading is the pinwheel with the middle four cells repeated beneath it, which is again of type 90°. Here are the pinwheel, that reading, and a six-cubie polycube of the same type.

⟨ Look at the type (v) picture itself 49 ⟩ ≡

```
pinwheel := []vec{{0, 2, 0}, {1, 0, 0}, {1, 1, 0}, {1, 2, 0},
    {2, 1, 0}, {2, 2, 0}, {2, 3, 0}, {3, 1, 0}}
twelve := append(append([]vec{}, pinwheel...), vec{1, 1, −1}, vec{1, 2, −1},
    vec{2, 1, −1}, vec{2, 2, −1})
mine := []vec{{0, 1, 0}, {1, 0, 0}, {1, 1, 0}, {1, 1, 1}, {1, 2, 0}, {2, 1, 0}}
for _, p := range [][]vec { pinwheel, twelve, mine }{
    fmt.Printf("␣␣%d␣cubies,␣connected␣%v,␣type␣%s:␣␣%s\n",
        len(p), connected(p), typeName(rot, rotIdx, rotCls, p), show(shift(p)))
}
```

This code is used in section 45.

50. The type (vi) drawing shows five top faces, and a bidiagonal polycube of six cubies can show five; so counting faces does not settle it. Their arrangement does. Fitting the four full ones to the projection puts them all at one height, in the shape of a T tetromino; the fifth is a sliver lower down. No bidiagonal polycube of six cubies, turned any of the 24 ways, has four exposed top faces at one height that make a T. Among those of seven cubies some do, and the only one that is not flat—a flat one would show all seven of its tops—shows exactly five: a T on one level, whose stem runs on into a column three cubies tall. That is almost certainly the picture, and it has seven cubies where six suffice.

The loop above leaves *cur* holding the polycubes of six cubies, so one more round of growth gives those of seven.

⟨ Look at the type (vi) picture itself 50 ⟩ ≡

```
for n := 6; n ≤ 7; n++ {
    if n ≡ 7 {
        ⟨ Add one cubie in every way 31 ⟩
    }
    ways, notFlat := 0, map[string]int{}
    for s := range cur {
        p := unkey(s)
        if typeName(rot, rotIdx, rotCls, p) ≠ "bidiagonal" {
            continue
        }
        for _, m := range rot {
            ⟨ Count m if it shows a T of exposed tops at one height 51 ⟩
        }
    }
    fmt.Printf("␣␣bidiagonal,␣%d␣cubies:␣%d␣orientations␣show␣a␣T␣of␣top␣faces" +
        "␣at␣one␣height", n, ways)
    for k, e := range notFlat {
        fmt.Printf(";␣not␣flat:␣%s,␣with␣%d␣exposed␣tops", k, e)
    }
    fmt.Println()
}
```

This code is used in section 45.

51. Four cells of a level make a T tetromino exactly when one of them has the other three as neighbours; so the level holds a T among its exposed tops exactly when one of them has at least three exposed neighbours.

⟨ Count m if it shows a T of exposed tops at one height 51 ⟩ ≡

```
q := make([]vec, len(p))
in := map[vec]bool{}
for j, c := range p {
    q[j] = act(m, c)
    in[q[j]] = true
}
level := map[int][]vec{}
exposed := 0
for _, c := range q {
    if ¬in[vec{c.x, c.y, c.z + 1}] {
        level[c.z] = append(level[c.z], c)
        exposed++
    }
}
isT := false
for _, cs := range level {
    for _, c := range cs {
        nb := 0
        for _, d := range cs {
            if abs(c.x − d.x) + abs(c.y − d.y) ≡ 1 {
                nb++
            }
        }
        if nb ≥ 3 {
            isT = true
        }
    }
}
if isT {
    ways++
    flat := true
    for _, c := range q {
        if c.z ≠ q[0].z {
            flat = false
        }
    }
    if ¬flat {
        notFlat[show(shift(q))] = exposed
    }
}
```

This code is used in section 50.

52. ⟨ Functions 5 ⟩ +≡

```
func abs(a int) int {
    if a < 0 {
        return -a
    }
    return a
}
```

53. Index.

a: 5, 9, 13, 14, 15, 17, 23, 24, 27, 40, 52.
abs: 51, 52.
act: 5, 10, 11, 25, 28, 29, 39, 41, 51.
again: 13.
all: 22.
answerOrder: 17, 18.
append: 6, 7, 13, 14, 25, 26, 29, 31, 35, 41, 42, 44, 49, 51.
ax: 10.
b: 5, 6, 9, 13, 14, 17, 23, 24, 27, 35, 40.
bar132: 10.
bent: 10.
best: 29, 47.
bits: 14, 15, 19, 22, 24.
Builder: 27.
byDet: 8.
byRule: 8.
c: 5, 11, 14, 19, 22, 24, 26, 27, 28, 29, 31, 33, 34, 35, 36, 37, 38, 43, 44, 47, 48, 51.
canon: 29, 30, 31, 46.
chosen: 42, 43.
classesOf: 14, 16, 19, 25.
closure: 13.
cls: 14, 33, 38, 42, 43.
comps: 8, 9.
conj: 15.
conjugate: 14.
connected: 43, 44, 49.
core: 37.
cs: 51.
cur: 30, 31, 32, 46, 50.
d: 31, 39, 41, 44, 51.
det: 5, 7, 10.
d4: 25, 26.
d4Cls: 25, 26.
d4Idx: 25.
e: 22, 50.
el: 13, 14, 15.
evens: 22.
exposed: 51.
exposedTops: 46, 47.
firstAt: 30, 32, 34.
firstIs: 30, 32, 34.
fits: 41.
fixes: 10, 11.
flag: 2.
flat: 51.
fmt: 8, 10, 19, 21, 22, 24, 26, 27, 29, 30, 34, 35, 36, 37, 45, 48, 49, 50.
found: 36, 37.
Fprintf: 27.
from: 42.
fullCls: 19.
fullIdx: 16, 19.
fullSubs: 19.
g: 13, 14, 26, 28, 33, 38, 39, 40, 41, 42, 43, 44.
grp: 12, 13, 14, 15, 26, 28, 33, 38, 40, 42, 44.
h: 13, 14, 15, 26, 33, 43.
have: 13.
i: 5, 6, 8, 9, 10, 13, 14, 15, 16, 18, 19, 22, 23, 24, 25, 26, 27, 28, 34, 36, 39, 41, 43, 48.
id: 15.
idx: 13, 14.
in: 11, 14, 31, 41, 44, 47, 51.
Int: 2.
inv: 5, 14.
invs: 8, 9.
isT: 51.
j: 5, 13, 23, 26, 28, 29, 42, 51.
Join: 26, 35.
k: 5, 13, 14, 15, 26, 29, 44, 47, 50.
key: 27, 28, 29.
least: 48.
left: 42.
len: 8, 13, 14, 17, 19, 24, 26, 27, 28, 29, 30, 33, 34, 36, 37, 40, 41, 42, 43, 44, 48, 49, 51.
level: 51.
lim: 2, 36.
lo: 27.
m: 6, 7, 8, 11, 15, 16, 22, 23, 24, 25, 28, 29, 39, 41, 50, 51.
main: 1.
make: 13, 27, 28, 29, 43, 51.
mat: 4, 5, 6, 7, 8, 10, 11, 13, 14, 15, 16, 23, 25, 28, 29, 33, 38, 40, 42.
min: 27.
mine: 49.
mode: 2, 3.
mostTops: 46, 48.
ms: 6, 7, 8, 16, 19.
mul: 5, 13, 14, 15.
n: 14, 18, 20, 23, 26, 30, 32, 34, 36, 38, 40, 41, 46, 48, 50.
nameOf: 15, 17, 19, 26, 33, 34, 36, 48.

names: 26.
nb: 51.
next: 31.
nm: 32, 34, 36, 46, 48.
notFlat: 50, 51.
ok: 32, 34, 39.
orb: 41.
orbitsAbout: 38, 40.
orbs: 38, 42.
order: 15.
out: 13, 27, 40, 41.
p: 6, 9, 11, 23, 24, 27, 28, 29, 31, 32, 33, 35, 38, 40, 41, 42, 43, 44, 46, 47, 49, 50, 51.
parityOf: 22, 23.
Parse: 2.
perms: 6, 7, 9.
pickOrbits: 38, 42.
pinwheel: 49.
printed: 45, 48.
Printf: 8, 10, 19, 22, 24, 26, 30, 34, 36, 37, 48, 49, 50.
Println: 8, 19, 21, 36, 45, 50.
q: 11, 28, 29, 31, 38, 39, 40, 41, 44, 51.
quarter: 15.
r: 5, 40, 41.
rc: 26.
roman: 19, 20, 34, 36, 48.
rot: 7, 8, 16, 17, 19, 22, 24, 25, 26, 28, 29, 30, 31, 32, 33, 34, 36, 38, 39, 40, 41, 42, 43, 46, 48, 49, 50.
rotCls: 16, 17, 19, 22, 24, 26, 32, 34, 36, 46, 48, 49, 50.
rotIdx: 16, 26, 28, 32, 33, 36, 38, 42, 43, 46, 49, 50.
rotSubs: 16, 19.
s: 6, 9, 18, 29, 31, 32, 46, 50.
same: 8.
sameClass: 43, 44.
seen: 14, 24, 40, 41, 44.
shift: 27, 28, 29, 43, 49, 51.
show: 34, 35, 36, 49, 51.
signed: 6, 7.
signs: 6, 7, 9.
Slice: 13, 14, 17, 27, 40.
smallest: 36, 38.
sort: 13, 14, 17, 26, 27, 40.
Split: 29.
Sprintf: 35, 48.
Sscanf: 29.
stack: 44.
String: 2, 27.
Strings: 26.
strings: 26, 27, 29, 35.
subgroups: 13, 14.
subs: 14.
symgroup: 28, 33, 43.
t: 11, 14, 29, 46.
ti: 14.
top: 2, 30, 34.
tr: 10.
TrimSuffix: 29.
twelve: 49.
twist: 10.
typeName: 32, 33, 46, 49, 50.
u: 47.
unkey: 29, 31, 32, 34, 46, 50.
v: 5, 10.
vec: 4, 5, 10, 11, 25, 27, 28, 29, 30, 31, 33, 35, 36, 38, 40, 41, 42, 43, 44, 46, 47, 49, 51.
verdict: 48.
w: 41.
want: 28.
ways: 50, 51.
where: 26.
x: 4, 5, 10, 11, 14, 27, 29, 31, 35, 37, 39, 40, 41, 43, 44, 47, 51.
y: 4, 5, 10, 11, 14, 27, 29, 31, 35, 37, 39, 40, 41, 43, 44, 47, 51.
z: 4, 5, 10, 11, 27, 29, 31, 35, 37, 39, 40, 41, 43, 44, 47, 51.

⟨ Add one cubie in every way 31 ⟩ Used in sections 30, 46, 50
⟨ Check the 24 rotations 8 ⟩ Used in section 3
⟨ Check the rotation the answer works out 10 ⟩ Used in section 8
⟨ Check what the answer says about particular types 21 ⟩ Used in section 3
⟨ Collect the orbit of p, if it is new and it fits 41 ⟩ Used in section 40
⟨ Count inversions and complementations of symmetry i 9 ⟩ Used in section 8
⟨ Count the subgroups and the symmetry types 19 ⟩ Used in section 3
⟨ Count m if it shows a T of exposed tops at one height 51 ⟩ Used in section 50
⟨ Do what the mode asks 3 ⟩ Used in section 1
⟨ Find the smallest polycube of each type 36 ⟩ Used in section 3
⟨ Find, for each size, the most top faces a polycube of each type can show 46 ⟩ Used in section 45
⟨ Functions 5 ⟩ Used in section 1
⟨ Give up on this centre unless the group acts about it 39 ⟩ Used in section 38
⟨ Grow every polycube and watch for new types 30 ⟩ Used in section 3
⟨ Look at the class of order 12 22 ⟩ Used in section 21
⟨ Look at the class of order 6 24 ⟩ Used in section 21
⟨ Look at the type (v) picture itself 49 ⟩ Used in section 45
⟨ Look at the type (vi) picture itself 50 ⟩ Used in section 45
⟨ Look for the core of eight answer 387 describes 37 ⟩ Used in section 36
⟨ Number the classes the way answer 387 does 17 ⟩ Used in section 16
⟨ Read the command line 2 ⟩ Used in section 1
⟨ Record any type seen for the first time 32 ⟩ Used in section 30
⟨ Report the smallest polycube of each type 34 ⟩ Used in section 30
⟨ Report these orbits if they make the polycube we want 43 ⟩ Used in section 42
⟨ Say where each type of square lands 26 ⟩ Used in section 25
⟨ Say which pictures are too big 48 ⟩ Used in section 45
⟨ Set up the two groups 7 ⟩ Used in section 1
⟨ Turn a square over 25 ⟩ Used in section 21
⟨ Types 4 ⟩ Used in section 1
⟨ Weigh the pictures in answer 387 45 ⟩ Used in section 3

Polycube symmetry types

	Section	Page
Introduction	1	1
The rotations of a cube	4	3
Subgroups, and symmetry types	12	7
What the answer says about three of the types	21	13
The symmetry type of a polycube	27	16
Growing all the polycubes	30	18
The minimum for every type	36	20
Weighing the pictures in answer 387	45	24
Index	53	30